

ProjectHub Backend + Frontend API/JWT/Annotation Reference

Prepared only from your backend and frontend folders. Base URL used in frontend: <http://localhost:8081>

Source Files Used

- Backend: backend/demo/demo/src/main/java/com/example/demo/controller/*.java
- Backend: backend/demo/demo/src/main/java/com/example/demo/security/JWTUtil.java and JWTFilter.java
- Backend: backend/demo/demo/src/main/java/com/example/demo/config/SecurityConfig.java
- Backend: backend/demo/demo/src/main/java/com/example/demo/service/UserService.java and ProjectService.java
- Frontend: frontend/projecthub/front/src/Login.jsx, ResetPassword.jsx, Dashboard.jsx, SubmitProject.jsx, Certificates.jsx, Profile.jsx, Notifications.jsx
- Frontend: FacultyDashboard.jsx, AssignedStudents.jsx, ReviewProjects.jsx, VerifyCertificates.jsx, FacultyNotifications.jsx
- Frontend: AdminDashboard.jsx, ManageUsers.jsx, AdminSettings.jsx, ViewSubmissions.jsx, AdminNotifications.jsx

Spring Boot Annotations Used With Explanation

Annotation	Source file(s)	Explanation
@SpringBootApplication	DemoApplication.java	Starts the Spring Boot application and enables auto-configuration/component scanning.
@RestController	All controller classes	Exposes backend methods as REST APIs that return JSON/text responses.
@RequestMapping	UserController, ProjectController, CertificateController, AdminController, etc.	Sets module base URL such as /users, /projects, /certificates, /admin.
@GetMapping	Controllers	Handles read APIs like dashboard load, project list, certificate list, profile load.
@PostMapping	Controllers	Handles create/action APIs like login, register, upload, create project/certificate, forgot password.
@PutMapping	Controllers	Handles update/review APIs like profile save, project review, certificate verify, settings update.
@DeleteMapping	Controllers	Handles delete APIs for users, projects, certificates, notifications, deadline rules.
@RequestBody	UserController, ProjectController, CertificateController, AuthController	Reads JSON body into DTO/entity/map.
@PathVariable	Controllers	Reads IDs in URL paths such as /users/{id}, /projects/download/{fileName}.
@RequestParam	Controllers	Reads query params and file upload field such as status, feedback, subject, file.
@CrossOrigin("**")	Controllers	Allows the React frontend to call backend APIs.
@Service	UserService, ProjectService	Marks business logic classes.
@Autowired	Controllers, services, security config/filter	Injects services, repositories, JWTFilter, JWTUtil, BCrypt encoder.
@Configuration / @Bean	SecurityConfig, PasswordConfig, OpenApiConfig	Defines Spring Security filter chain, password encoder, Swagger/OpenAPI bean.
@Component	JWTUtil, JWTFilter	Registers JWT helper and filter as Spring beans.
@Entity / @Table / @Id / @GeneratedValue	User, Student, Faculty, Admin, Project, AuditLog, AdminSettings	Maps MySQL database tables and primary keys.
@Document / @Id	ProjectDocument, CertificateDocument, NotificationDocument, DeadlineRule	Maps MongoDB collections/documents.
@OneToOne / @ManyToOne / @JoinColumn	Student, Faculty, Admin, Project, AuditLog	Defines entity relationships.
@Valid / @NotBlank / @NotNull / @Size	ProjectController, ProjectDocument	Validates project request data.

Annotation	Source file(s)	Explanation
@Transactional	UserService	Keeps multi-step user/profile delete/update work consistent.
@RestControllerAdvice / @ExceptionHandler	GlobalExceptionHandler	Formats backend exceptions into API error responses.
@Query / @Param	UserRepository	Supports custom normalized email lookup.
@Value	UserService	Reads app.frontend-url and mail username from application.properties.

JWT Workflow Explanation

- 1 Frontend Login.jsx encrypts the password using AES-GCM and sends POST /users/login with email and password payload.
- 2 UserController.login receives AuthRequest and calls UserService.login.
- 3 UserService normalizes email, finds user by normalized email, decrypts enc:v2 password when present, and verifies BCrypt password.
- 4 JWTUtil.generateToken creates an HS256 JWT with subject=email, role claim, issuedAt, and one-hour expiration.
- 5 Login.jsx stores token, userId, role, and fullName in localStorage and navigates based on role: STUDENT, FACULTY, ADMIN.
- 6 Frontend protected pages build headers with Authorization: Bearer <token> before fetch calls.
- 7 JWTFilter skips public routes: /users/login, /users/forgot-password, /users/reset-password, /auth/register, /landing/click, OPTIONS, Swagger, /error, and /projects/download/**.
- 8 For protected routes, JWTFilter validates the token, extracts role, creates ROLE_<ROLE>, and sets SecurityContextHolder.
- 9 SecurityConfig checks route rules: ADMIN for admin/settings/user management, FACULTY for review/verify APIs, STUDENT for submit/profile APIs, authenticated for common APIs.

Common Authentication + Button APIs

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Sign In button / Enter	Login.jsx handleLogin	POST	/users/login	AuthRequest {email,password}	Public	Returns token/id/role/fullName; frontend saves localStorage and navigates to role dashboard.
Forgot password? click	Login.jsx trackAuthAction	GET	/landing/click?action=forgot_password_click	action query	Public	Tracking call so the button click appears in Inspect/Network.
Back to Sign In click	Login.jsx trackAuthAction	GET	/landing/click?action=back_to_signin_click	action query	Public	Tracking call for forgot-mode back button.
Send Reset Link button	Login.jsx handleForgotPassword	POST	/users/forgot-password	{email}	Public	Creates reset token for STUDENT user and sends/returns reset link.
Open Reset Link button	Login.jsx trackAuthAction	GET	/landing/click?action=reset_password_link_click	action query	Public	Tracking call before navigating to /reset-password?token=...
Change Password button / Enter	ResetPassword.jsx handleChangePassword	POST	/users/reset-password	{token,newPassword,confirmPassword}	Public	Validates token and updates student password.
Reset Back to Sign In click	ResetPassword.jsx trackAuthAction	GET	/landing/click?action=reset_back_to_signin_click	action query	Public	Tracking call before navigate('/login').

Module 1 - Student APIs From Frontend Buttons/Screens

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Dashboard load	Dashboard.jsx	GET	/projects/analytics/student/{userId}	Bearer token	Loads total/approved/pending/rejected project counts.	
Dashboard load	Dashboard.jsx	GET	/projects/mongo/student/{userId}	Bearer token	Loads logged-in student's project submissions.	
Dashboard load	Dashboard.jsx	GET	/certificates/student/{userId}	Bearer token	Loads logged-in student's certificates.	
Dashboard load	Dashboard.jsx	GET	/users/student/profile/{userId}	Bearer token	Loads student profile summary.	
Export PDF button	Dashboard.jsx handleExportPDF	GET	same dashboard APIs again	Bearer token	Refreshes dashboard/project/certificate data before creating frontend PDF.	
Submit Project page load	SubmitProject.jsx	GET	/projects/mongo/student/{userId}	Bearer token	Submission history.	
Submit Project page load	SubmitProject.jsx	GET	/users/filter/role?role=FACULTY	Bearer token	Faculty dropdown.	
Submit Project page load	SubmitProject.jsx	GET	/deadline-rules/type/Project	Bearer token	Project deadline/resubmission rules.	
Submit Project button	SubmitProject.jsx handleSubmit	POST	/projects/upload	multipart file	Bearer token	Uploads selected project file.
Submit Project button	SubmitProject.jsx handleSubmit	POST	/projects/mongo	ProjectDocument JSON	Bearer token	Creates new project record.
Edit Project save	SubmitProject.jsx handleSubmit	PUT	/projects/mongo/update/{id}	ProjectDocument JSON	Bearer token	Updates existing project record.
Delete Project icon	SubmitProject.jsx handleDeleteProject	DELETE	/projects/mongo/{id}	id path	Bearer token	Deletes project.
View/download file button	SubmitProject.jsx handleDownloadProjectFile	GET	/projects/download/{fileName}	fileName path	Public in backend	Downloads/views uploaded project file.
Certificate page load	Certificates.jsx	GET	/certificates/student/{userId}	Bearer token	Student certificates.	
Certificate page load	Certificates.jsx	GET	/users/filter/role?role=FACULTY	Bearer token	Faculty dropdown.	
Certificate page load	Certificates.jsx	GET	/deadline-rules/type/Certificate	Bearer token	Certificate deadline/resubmission rules.	
Submit Certificate button	Certificates.jsx handleCertificateSubmit	POST	/projects/upload	multipart file	Bearer token	Uploads certificate file.
Submit Certificate button	Certificates.jsx handleCertificateSubmit	POST	/certificates	CertificateDocument JSON	Bearer token	Creates certificate record.

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Edit Certificate save	Certificates.jsx handleCertificateSubmit	PUT	/certificates/{id}	CertificateDocument JSON	Bearer token	Updates certificate.
Delete Certificate icon	Certificates.jsx handleDelete	DELETE	/certificates/{id}	id path	Bearer token	Deletes certificate.
View certificate file	Certificates.jsx handleViewCertificate	GET	latestCert.fileURL	file URL	Public/file route	Fetches certificate file blob.
Profile load	Profile.jsx	GET	/users/student/profile/{userId}	Bearer token	Loads profile form.	
Save Profile button	Profile.jsx handleSave	PUT	/users/student/profile/{userId}	profile JSON	Bearer token	Updates student profile.
Notifications page load	Notifications.jsx	GET	/users/notifications/{userId}	Bearer token	Loads notifications.	
Mark all read button	Notifications.jsx handleMarkRead	PUT	/users/notifications/read/{id}	id path	Bearer token	Marks notifications read.
Delete notification icon	Notifications.jsx handleDelete	DELETE	/users/notifications/{id}	id path	Bearer token	Deletes notification.

Module 2 - Faculty APIs From Frontend Buttons/Screens

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Faculty dashboard load	FacultyDashboard.jsx	GET	/projects/mongo/faculty/{userId}	Bearer token	Assigned projects and review counts.	
Faculty dashboard load	FacultyDashboard.jsx	GET	/certificates/faculty/{userId}	Bearer token	Assigned certificates and verification counts.	
Dashboard refresh/menu/stat clicks	FacultyDashboard.jsx fetchData	GET	same dashboard APIs	Bearer token	Refreshes faculty dashboard before/while navigating.	
Assigned Students page load	AssignedStudents.jsx	GET	/users/faculty/{facultyUserId}/students	Bearer token	Loads students assigned to faculty.	
Assigned Students page load	AssignedStudents.jsx	GET	/projects/mongo/faculty/{userId}	Bearer token	Loads assigned students' projects.	
Review Projects page load/filter click	ReviewProjects.jsx fetchProjects	GET	/projects/mongo/faculty/{userId}	Bearer token	Loads projects for review/filter list.	
Review Projects page load/filter click	ReviewProjects.jsx fetchProjects	GET	/deadline-rules/type/Project	Bearer token	Loads project rules.	
Approve/Reject Project buttons	ReviewProjects.jsx handleReview	PUT	/projects/mongo/review/{id}?status=&feedback=&grade=	query params	Bearer token	Approves/rejects project with feedback and grade.
Download Project file	ReviewProjects.jsx handle DownloadProjectFile	GET	/projects/download/{fileName}	fileName path	Public/file route	Downloads project file.
Verify Certificates page load/filter click	VerifyCertificates.jsx fetchCertificates	GET	/certificates/faculty/{userId}	Bearer token	Loads certificates assigned to faculty.	
Verify Certificates page load/filter click	VerifyCertificates.jsx fetchCertificates	GET	/deadline-rules/type/Certificate	Bearer token	Loads certificate rules.	
Approve/Reject Certificate buttons	VerifyCertificates.jsx handleCertificateReview	PUT	/certificates/{id}/verify?status=&remarks=	query params	Bearer token	Approves/rejects certificate with remarks.
Download certificate file	VerifyCertificates.jsx handleDownload	GET	fileURL	file URL	Public/file route	Downloads certificate file.
Profile load	Profile.jsx role=FACULTY	GET	/users/faculty/profile/{userId}	Bearer token	Loads faculty profile.	
Save Profile button	Profile.jsx handleSave	PUT	/users/faculty/profile/{userId}	profile JSON	Bearer token	Updates faculty profile.
Notifications page load	FacultyNotifications.jsx	GET	/users/notifications/{userId}	Bearer token	Loads notifications.	
Mark all read button	FacultyNotifications.jsx handleMarkAllRead	PUT	/users/notifications/read/{id}	id path	Bearer token	Marks notifications read.
Delete notification button	FacultyNotifications.jsx handleDelete	DELETE	/users/notifications/{id}	id path	Bearer token	Deletes notification.

Module 3 - Admin APIs From Frontend Buttons/Screens

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Admin dashboard load/refresh	AdminDashboard.jsx	GET	/users/admin/analytics	Bearer token	Loads total users/projects/certificates/faculty analytics.	
Admin dashboard load/refresh	AdminDashboard.jsx	GET	/users/audit-logs	Bearer token	Loads audit logs.	
Manage Users page load/refresh	ManageUsers.jsx fetchUsers	GET	/users	Bearer token	Loads all users.	
Edit icon profile fetch	ManageUsers.jsx inline onClick	GET	/users/student/profile/{user.id}	Bearer token	Loads student profile for edit modal.	
Edit icon profile fetch	ManageUsers.jsx inline onClick	GET	/users/faculty/profile/{user.id}	Bearer token	Loads faculty profile for edit modal.	
Delete user icon	ManageUsers.jsx handleDelete	DELETE	/users/{id}	Bearer token	Deletes user.	
Save Changes button	ManageUsers.jsx handleSave	PUT	/users/{id}	user/profile JSON	Bearer token	Updates user and role profile fields.
Add User Save Changes button	ManageUsers.jsx handleCreateUser	POST	/auth/register	user JSON	Public in config	Creates new user.
Add User Save Changes button	ManageUsers.jsx handleCreateUser	PUT	/users/{createdUser.id}	extra profile JSON	Bearer token	Saves additional role-specific fields after creation.
Bulk Upload button	ManageUsers.jsx handleBulkUpload	POST	/users/bulk-upload	multipart Excel file	Bearer token	Bulk registers users from Excel.
View Submissions page load/filter click	ViewSubmissions.jsx fetchData	GET	/admin/projects	Bearer token	Loads all project submissions.	
View Submissions page load/filter click	ViewSubmissions.jsx fetchData	GET	/admin/certificates	Bearer token	Loads all certificate submissions.	
Settings page load/refresh	AdminSettings.jsx	GET	/deadline-rules	Bearer token	Loads deadline rules.	
Settings page load/refresh	AdminSettings.jsx	GET	/users/audit-logs	Bearer token	Loads audit logs.	
Settings page load/refresh	AdminSettings.jsx	GET	/users/settings	Bearer token	Loads admin settings.	
Settings page load/refresh	AdminSettings.jsx	GET	/users/filter/role?role=FACULTY	Bearer token	Loads faculty dropdown.	
Save Settings button	AdminSettings.jsx saveSettings	POST	/users/settings	settings JSON	Bearer token	Saves admin settings.
Save Deadline button	AdminSettings.jsx saveDeadlineRule	POST	/deadline-rules	deadline JSON	Bearer token	Creates/updates deadline rule.
Assign Faculty button	AdminSettings.jsx handleAssignFaculty	PUT	/users/assign-faculty/{id}?subject=	query params	Bearer token	Assigns subject to faculty.
Assign Students button	AdminSettings.jsx handle AssignStudentsByDepartments	PUT	/users/admin/assign-students-by-department?department=§ion=&facultyUserId=	query params	Bearer token	Bulk assigns students by department/section.

Button/screen	Frontend source	Method	Backend API	Payload/params	Auth	Explanation
Admin notifications load	AdminNotifications.jsx	GET	/users/notifications/{userId}	Bearer token	Loads admin notifications.	
Mark all read button	AdminNotifications.jsx handleMarkAllRead	PUT	/users/notifications/read/{id}	id path	Bearer token	Marks notifications read.
Delete notification	AdminNotifications.jsx handleDelete	DELETE	/users/notifications/{id}	id path	Bearer token	Deletes notification.

Backend Controller Base Paths

Controller	Base path	Used for
UserController	/users	Login, forgot/reset password, users, profiles, settings, notifications, faculty assignment, student-faculty assignment.
AuthController	/auth	User registration.
ProjectController	/projects	File upload/download, project CRUD, review, search/filter, analytics.
CertificateController	/certificates	Certificate CRUD, resubmit, faculty verification.
DeadlineRuleController	/deadline-rules	Deadline rule list, by type, create/update, delete.
AdminController	/admin	Admin-only users/projects/certificates/dashboard APIs.
LandingController	/landing	Button click tracking APIs used by auth UI.
DashboardController	/dashboard	Dashboard count API.

Important Note About Button APIs

Buttons that only navigate pages, open modals, toggle dropdowns, show/hide passwords, choose filters, or close popups do not call backend APIs by themselves. They are UI-only actions. This PDF lists the buttons/screens that trigger fetch calls or backend-visible tracking calls in your frontend code.